

Building AI Application Challenge 2026

Builder Progress Log & Submission Workbook

Days 2-8 (Environment → Build → Evaluate → Deploy → Submit)

Use this document as a living workbook. Update it daily with links to your repo, demos, screenshots, and decisions.

0) Participant & Project Metadata

Participant Name	Nipun Kavinda
Email	Nipun.mecheng@gmail.com
Timezone	GST (Dubai)
Start Date	05 th June 2026
Path (select one)	● ☐ Model Building ● ☒ LLM/API Integration ● ☐ No-Code/Low-Code
Team Members (if any)	-
Mentor / Reviewer (if any)	-
Project Repo URL (GitHub)	Nipun Kavinda GitHub NovaDXB
Demo / App URL (if available)	Nipun Kavinda NovaDXB HuggingFace

Quick links (keep updated):

- ☒ GitHub repo (public link)
- ☒ Deployment link (Hugging Face / Streamlit / other)
- ☒ Demo video link (Loom/YouTube)
- ☒ LinkedIn progress post URL
- ☒ Tweet/X progress post URL (if applicable)

1) Day 1 Idea Summary (from your Day 1 template)

Fill this section by copying your Day 1 submission. Keep it crisp but specific.

Project Title	NovaDXB — AI Concierge Agent for Dubai Tourism
Concept Summary	NovaDXB is an AI-powered agentic concierge for Dubai tourists and visitors. Dubai attracts over 18M tourists annually, but many travelers rely on generic searches, outdated travel blogs, or costly human concierge services. NovaDXB fixes this by providing a personalized, actionable Dubai itinerary built around the user's travel style, budget, group type, and preferences. Powered by Retrieval-Augmented Generation (RAG) and a multi-tool agent workflow, NovaDXB proactively asks clarifying questions, retrieves grounded information from a curated Dubai knowledge base, and iteratively refines recommendations. It produces time-blocked day plans, suggests the best areas to stay, recommends dining and activities, and provides budget estimates—turning “where should I go?” into a complete plan you can act on immediately.
Target Audience	• International tourists visiting Dubai for the first time or planning a return visit • UAE residents planning staycations or hosting international guests • Repeat visitors who want “best-of” or hidden/alternate options • Busy travelers who need an itinerary quickly (not hours of browsing) • Families (kid-friendly pacing)
Key Features (top 3–5)	1. Agentic multi-turn itinerary planner 2. RAG-powered knowledge base 3. Itinerary cards + map visualization 4. Real-time budget dashboard 5. MCP tool integrations: Maps + Budget Estimator + Area Matcher; additional tools (Experience recommender, Weather, Currency converter) integrated where supported. 6. Premium luxury UI
Technical Approach (high-level)	• Knowledge Base: Pinecone (vector DB) • RAG: LlamaIndex, OpenAI Embeddings • Agent: LangChain (ReAct Agent), 7 custom MCP tools • Backend: Flask, Waitress • Frontend: HTML/CSS/JS (split-panel UI) • Deployment: Docker, HuggingFace Spaces
Expected Challenges	1. Knowledge base accuracy and coverage: Ensuring Dubai area/attraction data is accurate, well-structured, and semantically rich enough for good RAG retrieval. 2. Agent reliability and tool orchestration: The ReAct agent may hallucinate tool calls or produce inconsistent itinerary formats. 3. UI performance and real-time streaming: Making the right-panel itinerary update in real time as the agent streams its response.

	4. HuggingFace Spaces cold start latency: Free-tier Spaces can have slow cold starts during a live demo.
Expected Outcome (by Day 8)	By the end of the 8-day challenge, NovaDXB will be a fully deployed, production-quality AI concierge agent for Dubai tourism — accessible to anyone worldwide via a public HuggingFace Spaces URL. Technical outcome: A complete agentic RAG system with 7 MCP tools, a premium split-panel UI, real-time streaming itinerary generation, and a Dubai-specific knowledge base covering 20+ areas and 50+ experiences. Challenge outcome: Strong performance against judges' criteria through a clear, premium, working 60-second demo that demonstrates real-world value and technical depth. Portfolio outcome: A standout project for LinkedIn and GitHub that demonstrates end-to-end AI product development skills — from knowledge base design and RAG architecture to agentic tool orchestration, UI engineering, containerization, and cloud deployment.
Additional Notes	NovaDXB is strategically positioned to differentiate from last season's projects by targeting a universal-appeal domain (tourism) rather than a niche industrial domain — ensuring every judge immediately understands and relates to the product. The name NovaDXB combines 'Nova' (new, bright, explosive energy) with 'DXB' (Dubai's globally recognised airport code) — instantly communicating Dubai identity and technological innovation to any international audience. The project builds directly on proven architecture from DubaiNest AI (RAG + Pinecone + Flask + HuggingFace), meaning core infrastructure is already validated — allowing full focus on the agentic layer, MCP tools, and premium UI that will differentiate this submission.

2) Architecture Snapshot (living design)

Update this whenever your design changes.

One-line problem statement	Dubai tourists struggle to plan personalized trips — generic blogs and static lists don't account for individual budget, style, or real-time questions.
Primary user story / flow	User asks a natural-language question (e.g. "Plan 3 days in Dubai for a couple on mid budget") → agent reasons about which tool(s) to call → retrieves grounded context from the Dubai knowledge base → returns a specific, actionable response with real places and AED costs.
Inputs	User chat message (free text via web UI)
Outputs	Conversational response (markdown-formatted text) with itinerary, area/dining/budget recommendations
Key components (UI, backend, storage, etc.)	• UI: HTML/CSS/JS chat interface (split-panel) • Backend: Flask + Waitress (REST API) • Agent: LangChain React Agent + 7 MCP tools • RAG: LlamaIndex retrieval layer • Storage: Pinecone (vector DB), local KB files (CSV/TXT) • Deployment: Docker container on HuggingFace Spaces
Data flow diagram (link/image)	<pre> graph TD User[User] -- chat message --> Frontend[Frontend HTML / CSS / JS chat UI] Frontend -- POST /chat --> Flask[Flask backend Sanitized, rate-limited, cached] Flask --> Agent[LangChain ReAct agent] subgraph Agent Reasoning[Reasoning loop Thought, action, observation] Tools[7 MCP tools Itinerary, budget, area, dining, weather, currency, knowledge search] Reasoning --> Tools end Agent -- query --> RAG[RAG engine LlamaIndex retrieval + embeddings] RAG --> DB[Pinecone vector DB Dubai knowledge base, 7 sources] </pre>
Non-functional requirements (latency, cost, privacy, uptime)	• Latency: ~10-20s per agent response (multi-tool reasoning) (latency, cost, privacy, uptime) • Cost: OpenAI API usage per query (gpt-4o-mini, kept low via response caching) • Privacy: No PII stored; rate-limited per IP; input sanitised • Uptime: HuggingFace Spaces hosting (no formal SLA at hackathon scale)

3) Data / Knowledge Sources Inventory

Day 2 focus: list your data sources clearly. This becomes critical for quality, evaluation, and judging.

Source	Type	Owner / License	Format	Size	Update freq	Access method	Preprocessing needed	PII/Sensitive?	Status
Dubai Areas Guide	Curated text	Self-authored	CSV	~20 row	Manual, as needed	Local file → Pinecone	Row-to-text conversion	No	Complete
Dubai attractions	Curated text	Self-authored	CSV	~30 row	Manual, as needed	Local file → Pinecone	Row-to-text conversion	No	Complete
Dubai dining guide	Curated text	Self-authored	CSV	~25 row	Manual, as needed	Local file → Pinecone	Row-to-text conversion	No	Complete
Practical info (visa, transport, culture)	Curated text	Self-authored	Text	Single doc	Manual, as needed	Local file → Pinecone	None	No	Complete
Budget logic guide	Curated text	Self-authored	Text	Single doc	Manual, as needed	Local file → Pinecone	None	No	Complete
Seasonal/ weather guide	Curated text	Self-authored	Text	Single doc	Manual, as needed	Local file → Pinecone	None	No	Complete
Hidden gems / local tips	Curated text	Self-authored	Text	Single doc	Manual, as needed	Local file → Pinecone	None	No	Complete

If you are doing RAG / search:

- ☒ Corpus assembled (what content?)
- ☒ Chunking strategy defined (size/overlap)
- ☒ Embedding model selected
- ☒ Vector store selected (or file-based retrieval)
- ☒ Citation strategy (how will you show sources?)
- ☒ Data safety: remove secrets/PII and respect copyright

4) LLM / Model Selection & Experiments

Capture what you tried, what worked, and why you chose your final stack.

Candidate	Provider	Why considered	Prompting approach	Quality notes	Latency notes	Cost notes	Decision
gpt-4o-mini	OpenAI	Default/cheapest tier capable of reliable tool-calling in a LangGraph ReAct loop; natural baseline for a cost-sensitive concierge app	Single SystemMessage system prompt (see Section 2) + 7 @tool-decorated functions exposed via create_react_agent; temperature 0.7, max_tokens 1000	20/20 heuristic pass on Track 1 (real place names, AED prices, no vague filler, correct tool selection). On Track 2's 5-question subset: 5/5 pass, answers read as specific and well-organized (named restaurants/areas/hotels with price ranges)	Highest avg latency of the 3 models tested (15.49s avg on Track 2, vs 8-9s for the other two) — plausibly because it sometimes reasons through more ReAct steps before answering, even after the over-calling fix (see Section 3, prompt v2 notes)	Cheapest by a wide margin: \$0.000491 avg/query on Track 2 (vs \$0.001125 for gpt-3.5-turbo, \$0.007074 for gpt-4o — gpt-4o costs ~14x more per query)	✅ Selected as production default. Best cost/quality tradeoff for this use case; the latency gap (a few extra seconds) is acceptable for a chat concierge and not worth ~14x the cost of gpt-4o
gpt-4o	OpenAI	Premium comparison — wanted to know if the quality jump (if any) justified the cost, given NovaDXB markets itself as a "premium AI concierge"	Identical system prompt and tools, model string swapped at runtime (no other changes)	5/5 pass on Track 2's final run. Notable behavioral finding: on the first comparison run, gpt-4o asked a clarifying question instead of calling dining_recommender for an ambiguous dining request, while gpt-4o-mini and gpt-3.5-turbo both	Fastest avg latency of the 3 (8.38s) on the final run	Most expensive by far: \$0.007074 avg/query — roughly 14x gpt-4o-mini, ~6x gpt-3.5-turbo	❌ Not selected. No quality difference large enough (from heuristic + spot-read review) to justify a ~14x cost multiplier for this use case. Kept as a documented comparison point, not a production candidate

				proactively called the tool with reasonable defaults — this resolved after the prompt fix in Section 3, but is worth knowing as a documented behavioral difference between models, not just a bug			
gpt-3.5-turbo	OpenAI	"Budget" comparison — older/cheaper model, wanted to confirm whether it was actually meaningfully cheaper than gpt-4o-mini given it's nominally the "legacy" tier	Identical system prompt and tools, model string swapped at runtime	5/5 pass on Track 2's final run. Quality read as comparable to gpt-4o-mini on the questions tested, though manual 1-5 scoring (per-question, side-by-side) was not yet completed — see Section 4, Open Items	Mid-range latency (9.07s avg) — close to gpt-4o, faster than gpt-4o-mini	Not actually cheaper than gpt-4o-mini — \$0.001125 avg/query, roughly 2.3x gpt-4o-mini's cost despite being the "budget" option. This is a real, measured finding, not an assumption	✗ Not selected. No cost advantage over GPT-4o-mini was found in practice — the "budget" framing didn't hold up against actual measurement

Prompt versions (for LLM/API or No-Code tools): Not Started

Prompt ID	Goal	System constraints	Few-shot examples?	Tools/Actions used	Notes / Results
V1-Baseline	Original production SYSTEM_PROMPT in agent.py: establish persona, force tool use over general knowledge, require real place names and AED prices	"Always mention real names," "Always include AED prices when relevant," "Never answer from general knowledge alone"	None — instruction-only, no few-shot examples	All 7 tools (dubai_knowledge, itinerary_builder, budget_estimator, area_recommender, dining_recommender, currency_converter, weather_advisor)	Strong baseline — 18/20 on first clean Track 1 run (2 flags were eval heuristic false positives on visa/weather questions, not real agent problems — see Section 4). Found issue: dining_recommender was called repeatedly in one turn (3x on one Track 1 question, 4x on another, up to 6x on a Track 2 question) — agent split multi-area/multi-cuisine dining requests into one tool call per sub-topic instead of one combined call. Also observed gpt-4o (only, not gpt-4o-mini/gpt-3.5-turbo) asking a clarifying question instead of calling the tool on an ambiguous dining request
v2 — call-count constraint added	Fix the repeated-tool-call pattern found under v1, specifically on multi-part dining/area/itinerary requests	Added one paragraph: <i>"When a request spans multiple sub-topics (e.g. several areas, cuisines, or days), call the relevant tool once with all of those sub-topics combined into</i>	None	Same 7 tools, no tool-level changes	Confirmed fix, verified by direct before/after comparison on the same question (C04, dining): gpt-4o-mini dropped from 6 tool calls (3,786 tokens, \$0.000928, 19.66s) to 1 tool call (2,161 tokens, \$0.000553, 16.88s) — a ~40% cost reduction on that specific question.

		<i>a single input... Synthesize variety from one tool response instead of making repeated calls to the same tool in one turn."</i> Everything else in v1 unchanged — no tool docstrings, no tool logic, no other prompt text touchedFix the repeated-tool-call pattern found under v1, specifically on multi-part dining/area/itinerary requests			Side effect (welcome, not fully predicted): gpt-4o's clarifying-question dodge also disappeared — it began calling the tool directly like the other two models. Re-ran the full 3-model comparison after this change: 5/5 pass across all 3 models, every question called its expected tool exactly once
--	--	---	--	--	---

Model-building path only (if applicable):

Framework	N/A
Model family	N/A
Baseline approach	N/A
Training data	N/A
Validation strategy	N/A
Key metrics	N/A

5) Day 2 — Environment Setup & Initial Development

Deliverable for Day 2: no full project submission today—prepare your environment + tech stack/tooling + initial work evidence.

Environment setup checklist (tick what applies):

- ☒ Repository created and initial commit pushed
- ☒ README created (project goal + how to run)
- ☒ .gitignore configured (no secrets)
- ☒ Python environment created (venv/conda) OR platform workspace created (no-code)
- ☒ Dependencies installed and pinned (requirements.txt / pyproject / lockfile)
- ☒ API keys stored safely (.env, secrets manager, or platform secrets)
- ☒ Basic 'hello world' run completed locally
- ☒ Basic API call tested (if using LLM/API) OR first workflow run (if no-code)
- ☒ Folder structure created (src/, data/, notebooks/, etc.)
- ☒ First prototype screen/flow created (UI stub acceptable)

Evidence (links):

GitHub repo URL	Nipun Kavinda GitHub NovaDXB
Commit hash / tag	Initial Commit
Screenshot(s) link	Google Drive Day 2
Notes on setup decisions	• Python version: 3.10+ is used for compatibility with all libraries. • Virtual environment: venv chosen over conda — lighter weight, no additional install required, easier to reproduce on any machine. • LLM choice: OpenAI GPT-4o mini selected as primary. API key stored in .env file using python-dotenv. Key is listed in .gitignore so it is never committed to GitHub.

Initial development log (what you built today):

- Set up a Python virtual environment (venv) for the project
- Installed initial dependencies via requirements.txt (Flask, LangChain, LlamaIndex, Pinecone)
- Built initial project folder structure (data/, static/, root-level files)
- Created GitHub repository NovaDXB and confirmed version control is active
- Created .env file for API key management (OpenAI, Pinecone)
- Create Initial Chat UI

Blockers/issues encountered (and how you resolved them):

Nothing Special for Day 2

Day 3 — Building the Brain of Your App

Checklist:

- ☒ Core logic implemented (model/prompt/workflow)
- ☒ Data ingestion or API integration expanded beyond hello world
- ☒ 10–20 test questions/examples drafted (start your 'exam set')
- ☒ First measurable baseline created (even if rough)
- ☒ README updated with run instructions

Artifacts / notes:

What is the 'brain' of your app (1–2 sentences)?	An AI agent that reasons about a traveler's request, selects from a set of specialized tools, and retrieves grounded context from a curated Dubai knowledge base to generate specific, actionable recommendations — rather than relying on general LLM knowledge alone.
Link to key code/workflow	Nipun Kavinda GitHub NovaDXB
Baseline results (short)	Manually tested with sample queries (e.g. "plan a 3-day Dubai trip for a couple on mid budget") agent correctly selects relevant tools and returns grounded, specific responses referencing real places and costs from the knowledge base. No formal accuracy benchmark yet.
What you will improve next	Build a structured test set of 10-20 example queries to evaluate consistency, add explicit source citation in responses, and begin tracking response quality more systematically.

Decisions made today (why):

- Used a curated, self-authored knowledge base instead of scraping external sources — ensures content quality and avoids licensing/ copyright issues for a fast-moving challenge timeline
- Chose tool-based agent reasoning over a single large prompt — keeps each capability (budget, dining, itinerary, etc.) modular and easier to test independently

Blockers / help needed:

No formal test set yet — need to draft 10-20 example queries to properly baseline response quality before adding new features

Day 4 — Optimizing Integration & Application Evaluation

Checklist:

- ☒ Evaluation approach defined (metrics + test set)
- ☒ Error handling + retries added (API) OR validation rules added (no-code)
- ☒ Prompt/model iteration based on failures
- ☒ Latency/cost notes captured
- ☒ Safety/guardrails considered (content, PII, injection)

Artifacts/notes:

Evaluation dataset link/location	Nipun Kavinda Day 4 Google Drive
Metrics used (accuracy, faithfulness, etc.)	Manual review of response quality against knowledge base content (groundedness — does it reference real places from the KB?) Will formalise with test set tomorrow
Top failure modes found	1. Early responses returned vague answers ("mid-budget area") instead of real Dubai neighbourhood names 2. CSV files failed to load due to comma-in-field parsing errors 3. LangChain/LangGraph import paths broke after version mismatch between langchain 1.3.4 and langgraph-prebuilt 1.1.0
Fixes applied	1. Added explicit system prompt instructing GPT to always use real names from retrieved context — fixed response quality 2. Switched CSV loading to pandas with engine="python" and on_bad_lines="skip" all 7 KB files now load correctly 3. Force-reinstalled full langgraph stack to resolve version mismatch — confirmed with isolated import test

Decisions made today (why):

- Chose itinerary extraction as a separate lightweight follow-up LLM call rather than modifying the agent's main output format — keeps the agent reasoning intact while adding structured side panel data without risking response quality degradation
- Used a quick heuristic check ("day 1" in response text) before making the extraction call — avoids unnecessary API costs for non-itinerary queries like weather or currency questions

Blockers / help needed:

Evaluation dataset not yet created — need to draft and run 10-20 structured test questions tomorrow to formally baseline response quality

Day 5 — Integration of Model/API with Interface

Checklist:

- ☒ User flow designed (screens + inputs/outputs)
- ☒ Interface connected to backend/logic
- ☒ UX basics: loading, error states, reset/clear
- ☒ Logging of inputs/outputs enabled (safe logging)
- ☒ Demo link created (even if rough)

Artifacts / notes:

Interface tech (Gradio/Streamlit/React/No-code UI)	HTML, CSS, JS
Demo link	Nipun Kavinda NovaDXB Huggingspace
Screenshots link	Nipun Kavinda Google Drive Day 5
Known UX issues to fix	1. Suggestions bar disappears after first message (currently hidden — better to keep as compact row) 2. The side panel does not reset/clear when the user starts a completely new, unrelated conversation 3. No explicit "clear chat" or "start over" button (UX gap for returning users)

Decisions made today (why):

- Choose a split-panel layout (chat left, itinerary right) over a single-column chat — judges and users can see the itinerary building in real time without scrolling through the full response, which makes the agentic behavior visually obvious during a demo
- Used a separate lightweight LLM extraction call (temperature=0) to convert the agent's markdown response into structured JSON for the side panel, rather than changing the agent's output format — this preserves the quality of the conversational response while adding structured UI data without risking regression in the agent's reasoning behavior

Blockers / help needed:

The side panel currently has no "clear/reset" mechanism — if a user asks multiple unrelated questions, old itinerary cards persist even when not relevant. Need to add a reset button or auto-clear logic for the next session

Day 6 — Final Enhancements, Security & Debugging

Checklist:

- ☒ Input validation + sanitization
- ☒ Secrets handling reviewed (no keys in repo)
- ☒ Rate limits / caching considered
- ☒ Bug list triaged and reduced
- ☒ README + architecture notes cleaned up

Artifacts / notes:

Security checklist notes	• Input sanitization: control chars stripped, length capped, JSON validation enforced on every /chat request . • Prompt-injection: 7 pattern regex guard, redirects to safe canned response without reaching the LLM. • Secrets: .env gitignored, .env.example committed, HF Space Secrets for production zero keys in repo. • CORS: restricted to known origins (localhost + HF Space URL) not wildcard * • SECRET_KEY: enforced at startup, no hardcoded fallback. • Rate limiting: 15 req/60s per IP, 429 response on exceed • Error handling: no raw exceptions leaked to client, full details logged server-side only via logging module
Top bugs fixed	• CSV parsing failure — 2 of 7 KB files silently skipped on first ingest due to commas inside text fields. Fix: pandas engine="python", on_bad_lines="skip" • Vague RAG responses — agent returning "mid-budget area" instead of real neighborhood names. Fix: explicit system prompt grounding responses to real KB name • Cached response missing itinerary — cached /chat path returned response without itinerary JSON for side panel. Fix: added extraction call to cached branch too • Extraction call wasted on non-itinerary responses — unnecessary API cost on weather/currency queries. Fix: heuristic "day 1" check skips extraction entirely
Remaining risks	In-memory cache/rate-limiter resets on app restart (acceptable at hackathon scale, would move to Redis in production)

Decisions made today (why):

- Kept rate limiter and cache in-memory rather than adding Redis — Redis adds a dependency and deployment complexity that isn't justified at hackathon scale. The in-memory solution directly addresses the reviewer's concern ("no rate limiting, no caching") without over-engineering for a demo environment
- Kept CORS restricted to two specific origins rather than using a dynamic allow-list — NovaDXB has exactly two deployment targets (localhost dev + HF Space), so a static list

is simpler, more auditable, and less likely to be misconfigured than env-var-driven origin management.

- Built and ran the evaluation harness referenced as tomorrow's blocker: 20 structured test questions across all 7 agent tools (Track 1, baseline gpt-4o-mini) plus a 5-question, 3-model comparison (gpt-4o-mini / gpt-4o / gpt-3.5-turbo — Track 2). Final results: 20/20 pass on Track 1, 5/5 pass across all three models on Track 2. Full results and tables in `Day4_Evaluation_Report.md`.

Blockers / help needed:

No formal SLA on HuggingFace Spaces free tier — cold start possible if Space sleeps

Day 7 — Final Review & Deployment

Checklist:

- ☒ Deployment target chosen and deployed
- ☒ Environment variables set in deployment platform
- ☒ Smoke tests run on deployed version
- ☒ Performance checked (latency/cost)
- ☒ Submission package checklist started

Artifacts / notes:

Deployment platform	Docker + Hugging Face Spaces
Deployment URL	Nipun Kavinda NovaDXB HuggingFace
Smoke test results	✓ /health endpoint → {"status": "ok", "app": "NovaDXB", "version": "1.0"} ✓ /chat POST → returns itinerary response with structured JSON ✓ Landing page (/) → renders correctly ✓ Chat UI (/app) → renders correctly, split panel working ✓ Markdown rendering → headers, bold, lists display correctly ✓ Side panel → itinerary cards build on itinerary queries ✓ Non-itinerary queries → side panel stays, no errors ✓ Prompt injection test → safely redirected, no LLM reached ✓ Rate limiting → 429 returned after threshold exceeded ✓ Empty message → 400 returned with clear error message
Fallback plan if deployment breaks	1. Check HF Space logs for startup errors (Settings → Logs in HF Space dashboard) 2. Verify all 5 HF Secrets are set correctly (OPENAI_API_KEY, PINECONE_API_KEY, PINECONE_INDEX_NAME, SECRET_KEY, INGEST=false) 3. If agent fails → revert to rag_engine.py only mode (comment out agent import, use query_rag directly in app.py) 4. If HF Space won't start → run locally on laptop for demo (python app.py — confirmed working locally throughout challenge) 5. If Pinecone unreachable → check Pinecone dashboard for index status and API key validity

Decisions made today (why):

Chose HuggingFace Spaces over other platforms (Railway, Render, Fly.io) — zero cost, Docker support, no sleeping issue during active challenge week, same deployment workflow as previous projects (DubaiNest AI, DORA) meaning zero learning curve

Kept Waitress as the WSGI server rather than Gunicorn — Waitress is pure Python, installs cleanly on any platform without C dependencies, and has been validated in production across multiple previous projects in this tech stack

Blockers / help needed:

HuggingFace free tier has no uptime SLA — Space may cold-start if inactive for extended periods

Day 8 — Final Submission & LinkedIn Sharing

Checklist:

- ☒ Final app link working
- ☒ Final repo is clean + documented
- ☒ Demo video recorded
- ☒ Final submission form completed
- ☒ LinkedIn post published + link shared

Artifacts / notes:

Final app URL	Nipun Kavinda NovaDXB HuggingFace
Final repo URL	Nipun Kavinda GitHub NovaDXB
Demo video URL	Nipun Kavinda Google Drive Presentations
LinkedIn post URL	Nipun Kavinda LinkedIn
Tweet/X URL (if applicable)	Nipun Kavinda X NovaDXB
Reflection: what you learned	Building NovaDXB taught me how to move from building simple AI prototypes to designing a full production-grade AI system. I learned that real AI applications depend not only on LLMs, but on data design, retrieval systems, agent orchestration, and user experience together. I gained experience in building an Agentic RAG system, using tools like LangChain, LangGraph, LlamaIndex, and Pinecone to improve reasoning and reduce hallucinations through structured knowledge bases. I also learned the importance of data curation over model reliance, as well as adding security layers, evaluation pipelines, and cost optimization to make the system reliable and efficient. On the product side, I understood how UX design (interactive itinerary, map integration, conditional UI) plays a key role in making AI systems practical and user-friendly. Overall, this project helped me understand the complete lifecycle of an AI product—from architecture and data design to deployment and evaluation.

Decisions made today (why):

- Used a **static 38-place Dubai landmark lookup table** instead of geocoding APIs → to avoid external dependency, reduce cost, and improve reliability
- Implemented **conditional “ Show map” button rendering** → to avoid empty/meaningless UI states and improve user experience.
- Used **Leaflet + OpenStreetMap tiles (no API key)** → to keep the system lightweight, free, and production-ready for deployment constraints.
- Today I also worked on the **Product introduction/Demo flow**.

Blockers / help needed:

- Improving **place recognition accuracy** across the full system, especially handling ambiguous or varied landmark names in user queries and itinerary text.
- Scaling beyond a fixed **38-location knowledge base** without increasing false matches; need smarter NLP or embedding-based retrieval for better generalisation.
- Exploring **embedding-based place detection or hybrid retrieval models** instead of strict text matching for higher robustness.
- Current system is **chat-only UI**; needs evolution into a more structured **application-style interface (forms, calendar selection, multi-step inputs alongside chat)** for better UX.
- Future expansion ideas include adding **hotel booking, ticket booking, flight/air ticket agent, and visa information assistant modules**, which will require multi-agent orchestration and external API integrations.
- UI/UX enhancements needed to transform from a conversational prototype into a **full travel application experience with guided workflows and state-based screens**

10) Community Sharing Tracker (bonus points)

Use this to track your social proof and claim points.

[illegible]

11) Reference Resources (from the challenge)

Core challenge walkthrough video: <https://www.youtube.com/watch?v=X4PitcxNDjE>

Python environment setup: <https://www.youtube.com/watch?v=D5XyQ96EgiM&t=759s>

GitHub basics: https://youtu.be/bV_9mr5O5bg?si=0oiiT9BZutcKCjnL

OpenAI API key setup: <https://www.youtube.com/watch?v=CVnTzj-qhCU&t=8s>

Model/LLM selection blogs:

- <https://decodingdatascience.com/choosing-the-right-gemini-model-for-your-ai-project-a-beginner-friendly-guide/>
- <https://decodingdatascience.com/openai-guide-to-use-which-model-for-tasks/>
- <https://decodingdatascience.com/how-to-choose-the-right-openai-model-gpt-5-complete-guide/>

AI Explorer RSVP: <https://nas.io/artificialintelligence/events/ai-explorer-ai-demos-ai-use-cases-and-q-a-1767887350300>

12) Reviewer / Mentor Notes (optional)

Use this section if an instructor/mentor is reviewing your progress.

Strengths observed:

-

Areas to improve next:

-

Action items:

-